Analysis Report — Cross-Review (Student A: InsertionSort, Student B: SelectionSort)

Deliverables included
- Implementations: `algorithms.InsertionSort`, `algorithms.SelectionSort`
- Tests: unit tests + property-based random checks (JUnit 5)
- Benchmarks: `metrics.PerformanceTracker` writes `performance_results.csv`
- Plots: docs/performance-plots/*.png
- Synthetic example CSV: docs/performance-plots/example_results.csv

---

1. Asymptotic Complexity Analysis

Insertion Sort (Student A)
- Time complexity:
  - Best (Ω): Ω(n) — when array already sorted; each insertion does one comparison.
  - Average (θ): θ(n^2)
  - Worst (O): O(n^2) — reverse-sorted array.
- Space complexity:
  - In-place, auxiliary O(1) beyond input array.
- Notes:
  - Binary insertion reduces comparisons to O(n log n) but shifts remain O(n^2).

Selection Sort (Student B)
- Time complexity:
  - Best/Average/Worst: θ(n^2) comparisons (selection does full scan each iteration).
- Space:
  - In-place, O(1).
- Notes:
  - SelectionSort performs O(n) swaps; good when swaps are expensive but comparisons are cheap.

---

2. Code Review & Optimization Suggestions

InsertionSort
- Strengths:
  - Simple, readable implementation, metrics recorded (comparisons, swaps, arrayAccesses).
  - Good handling of null and small arrays.
- Issues / Bottlenecks:
  - Counting conventions: 'swaps' used to count shifts and insertions — clarify naming (shifts vs swaps).
  - For large arrays, O(n^2) will be slow; suggest switching to TimSort/Arrays.sort for n > threshold.
- Improvements:
  - Add binary insertion mode to reduce comparisons for large keys.
  - Consider using System.arraycopy for shifting blocks, which is optimized in JVM.

SelectionSort
- Strengths:
  - Metrics and API compatible.
- Issues:
  - Array access counting assumes 2 accesses per comparison; minor inconsistency with insertion counting.
  - Could early-terminate if the remainder is already sorted? (Not for selection sort.)
- Improvements:
  - For small arrays selection sort is fine; otherwise replace with O(n log n) sort.

---

3. Empirical Validation (Plan + Synthetic Results)
Benchmark plan:
- Sizes: 100, 1,000, 10,000, 100,000
- Distributions: random, sorted, reverse_sorted, nearly_sorted
- Warm-up: 3 runs; Reps: 5; record median time; measure heap usage difference.

Synthetic results available at `docs/performance-plots/example_results.csv`. Plots (synthetic) included in `docs/performance-plots/`.

(When you run the real `metrics.PerformanceTracker` on your machine, `performance_results.csv` will be produced.)

---

4. Testing
- Unit tests provided for both algorithms, covering empty, single, duplicates.
- Property-based tests implemented via randomized arrays asserting algorithm outputs equal `Arrays.sort()` results.
- Cross-validation: Tests compare sorted output with standard `Arrays.sort()` in random property checks.

---

5. Optimization Impact
- Recommendations (to be validated):
  - Replace insertion/selection for large n with `Arrays.sort()` or MergeSort.
  - Use `System.arraycopy` to shift blocks in insertion — can reduce time.
- To measure: run benchmarks before/after and compare median times per size/distribution.

---

6. Reproducibility & Git workflow
- Branching: use feature branches, merge into `main`; tags for releases (`v0.1`).
- `metrics.PerformanceTracker` writes reproducible CSV; include seed if strict reproducibility required.

```
## 7. Next steps
1. Run `mvn test` locally and `java -cp target/classes cli.BenchmarkRunner` to produce real metrics.
2. Provide partner's test criteria; I'll run cross-tests and produce the final PDF (8 pages max) and pair summary.
```